

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет
по Лабораторной работе №3
«Контейнеризация написанного приложения средствами Docker»

Выполнил:
Суворин Иван
БР 2.2

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задание

- Реализовать Dockerfile для каждого сервиса;
- Написать общий docker-compose.yml;
- Настроить сетевое взаимодействие между сервисами.

Ход работы

2.1. Что контейнеризировано

4 собственных сервиса (gateway, auth-service, recipe-service, social-service) – по одному Dockerfile на каждый. RabbitMQ не контейнеризирован осознанно: в задании речь о «своих» сервисах, а RabbitMQ – внешняя инфраструктура, уже работающая на CloudAMQP с этапа 7.

2.2. Многоступенчатая сборка

Каждый Dockerfile состоит из двух стадий: build (ставит все зависимости, включая dev, компилирует TypeScript в dist/) и финальная стадия (только продакшн-зависимости + готовый dist/). Итоговый образ не содержит исходники и инструменты разработки.

Все зависимости проекта (sql.js, bcryptjs, amqplib) – чистый JavaScript без нативной компиляции, поэтому alpine-образ собирается без установки дополнительных системных пакетов (gcc, python и т.п.), которые обычно требуются для нативных модулей типа better-sqlite3 – от него отказались еще в ЛР1 по другой причине, и это решение теперь окупается второй раз.

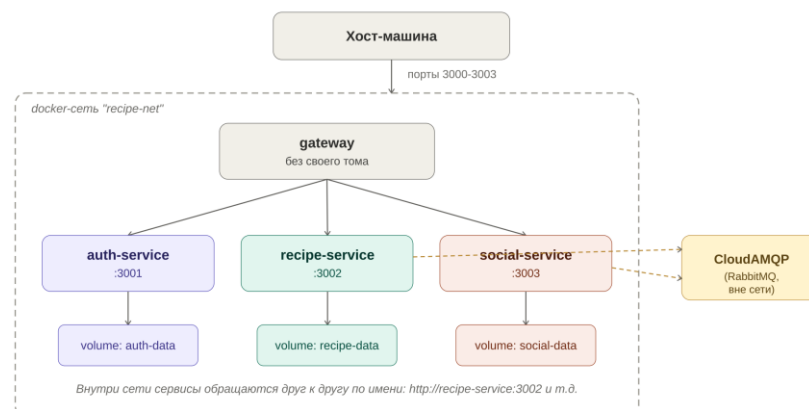


Рисунок 1 – Сеть, тома и внешний брокер сообщений

2.3. Особый случай: Gateway

Gateway – единственный сервис, чей Dockerfile собирается с контекстом = корень всего проекта (а не своя папка), потому что ему нужен файл `орепарі.yaml`, лежащий выше папки `services/`. В `docker-compose.yml` это указано явно:

```
Gateway:
  build:
    context: .
    dockerfile: services/gateway/Dockerfile
```

Путь к `орепарі.yaml` внутри контейнера передается через переменную окружения `OPENAPI_PATH`, а не вычисляется по количеству уровней `../../` – это позволяет одному и тому же коду одинаково работать и при обычном запуске (`ts-node-dev`), и в Docker, где структура папок другая.

2.4. Сетевое взаимодействие

Все 4 контейнера – в одной пользовательской docker-сети `recipe-net`. Внутри нее Docker предоставляет DNS по имени сервиса: `recipe-service` обращается к Auth Service как `http://auth-service:3001`, а не `http://localhost:3001`. Код при этом не менялся ни строчкой – адреса сервисов были вынесены в переменные окружения еще на этапе 6, здесь просто подставлены другие значения через `environment`: в `docker-compose.yml`.

2.5. Хранение данных

У каждого из трех сервисов с собственной БД – именованный том (`auth-data`, `recipe-data`, `social-data`), примонтированный на `/app/data` внутри контейнера. Без этого база пересоздавалась бы пустой при каждом `docker compose up`. Gateway тома не требует – он не хранит данных.

3. Возникшие сложности и их решение

3.1. Установка Docker Desktop на Windows 10

Docker Desktop не был установлен заранее. Установка WSL2 (`wsl --install`) завершилась ошибкой `0x800705aa` («недостаточно системных ресурсов») – по независимым источникам эта ошибка типична при нехватке места на диске или RAM. Решение в два шага: во-первых, вместо полной установки Ubuntu использован флаг `--no-distribution` (Docker Desktop не нуждается в пользовательском Linux-дистрибутиве, только в платформе WSL2 – это подтверждено официальной документацией Docker); во-вторых,

освобождено место на диске С стандартной очисткой (диск был заполнен почти полностью, оставалось 18 ГБ).

3.2. Docker Desktop не запускался – та же ошибка на другом шаге

После установки тот же код ошибки (0x800705aa) повторился при первой попытке `docker compose up --build` – на этот раз при создании внутренней рабочей машины самого Docker Desktop. Причина та же (нехватка места), решение то же – дополнительная очистка диска. После нее Docker Desktop поднялся, и сборка образов прошла успешно.

3.3. Циклическая зависимость в docker-compose.yml

Первый запуск `docker compose up` упал с ошибкой `dependency cycle detected: recipe-service -> social-service -> recipe-service`. Причина – в исходном `docker-compose.yml` `recipe-service` был указан как зависящий от `social-service`, и одновременно `social-service` – от `recipe-service`. `depends_on` описывает только порядок запуска контейнеров, а эти два сервиса действительно обращаются друг к другу в обе стороны во время работы – но одновременно стартовать друг после друга невозможно. Решение – убрать эту взаимную зависимость: она и не требовалась, так как оба сервиса уже устойчивы к временной недоступности соседа (заложено еще в ЛР2/Д35).

3.4. Ошибка строгой проверки типов в Gateway

Сборка Gateway падала на `npm run build`: `error TS2339: Property 'originalUrl' does not exist on type 'IncomingMessage'`. Код (`pathRewrite: (_path, req) => req.originalUrl`), написанный на этапе 6, работал в режиме разработки (`ts-node-dev --transpile-only` пропускает часть проверок типов), но не проходил полную компиляцию, которая используется в Docker-сборке. Библиотека `http-proxy-middleware` описывает `req` обобщенным типом `Node.js` без поля `originalUrl`, хотя на практике (внутри Express) оно всегда присутствует. Решение – явное приведение типа: `(req as Request).originalUrl`.

3.5. sql.js и порядок seed/restart в контейнерах

После первого успешного `docker compose up` тесты Postman провалились полностью: логин администратора возвращал 401, создание рецепта – 400. Причина та же, что уже встречалась при обычном (не-Docker) запуске в ЛР2: `sql.js` загружает базу в память процесса один раз при старте. Контейнеры `auth-service` и `recipe-service` были запущены (с пустой базой в памяти) ДО выполнения `docker compose exec ... node dist/seed.js` – сиды записались на диск, но уже работающий процесс их не увидел. Решение – `docker compose restart auth-service recipe-service` после сидирования, чтобы процессы перечитали заполненную базу с диска.

4. Демонстрация работы

Итоговая последовательность команд для чистого запуска:

```
docker compose down
docker compose up --build -d
docker compose exec auth-service node dist/seed.js
docker compose exec recipe-service node dist/seed.js
docker compose restart auth-service recipe-service
```

Проверка `docker compose ps` показала все 4 контейнера в статусе Up, с корректно опубликованными портами:

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
recipe-service-auth-service-1	recipe-service-auth-service	"docker-entrypoint.s..."	auth-service	47 seconds ago	Up 17 seconds	0.0.0.0:3001->3001/
tcp, [::]:3001->3001/tcp						
recipe-service-gateway-1	recipe-service-gateway	"docker-entrypoint.s..."	gateway	47 seconds ago	Up 45 seconds	0.0.0.0:3000->3000/
tcp, [::]:3000->3000/tcp						
recipe-service-recipe-service-1	recipe-service-recipe-service	"docker-entrypoint.s..."	recipe-service	47 seconds ago	Up 17 seconds	0.0.0.0:3002->3002/
tcp, [::]:3002->3002/tcp						
recipe-service-social-service-1	recipe-service-social-service	"docker-entrypoint.s..."	social-service	47 seconds ago	Up 46 seconds	0.0.0.0:3003->3003/
tcp, [::]:3003->3003/tcp						

Критерий успеха – та же Postman-коллекция из ДЗ3, без единой правки, прогнанная против контейнеризированной системы (`http://localhost:3000`):

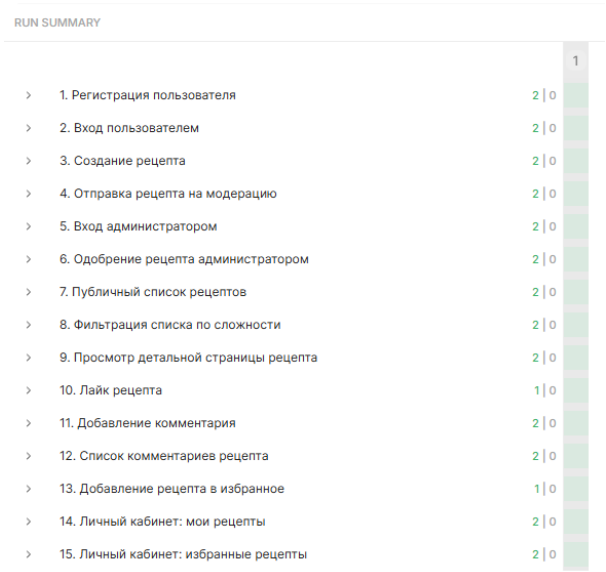


Рисунок 2 – Итоговый прогон: все 15 запросов и 28 тестов пройдены (0 failed)

ВЫВОД

Все 4 сервиса контейнеризированы через многоступенчатые Dockerfile, объединены общим docker-compose.yml в одну сеть с обнаружением друг друга по имени контейнера, данные сохраняются в именованных томах, секреты – в едином .env. Внешний контракт API не изменился: та же Postman-коллекция из ДЗ3 проходит все 28 тестов без единой правки, что подтверждает корректность контейнеризации.